

Krickora - Self-Service Keypad Entry (Raspberry Pi build)

DIY Build Plan & Specification - Rev 0.2 (draft) - 2026-06-05 - Jaycar (AU) sourcing **Controller variant:** Raspberry Pi 4 (supersedes the ESP32 controller in keypad-build-plan.md)

Read first - assumptions. Reconstructed from the booking/access-code software (src/lib/access-code.ts, Convex + Google Calendar + Stripe) and our design discussion. Items still to be decided are marked **[DECISION]**. Confirm flagged items before ordering/installing. Prices are indicative AUD only - verify current Jaycar pricing/stock. **This is a draft, not a certified design** - egress/fire compliance needs licensed sign-off (§13).

1. What changed from the ESP32 plan

Everything except the controller is **identical** and carries over unchanged:

- Door strike (LA5077), IR exit switch (LA5187), keypad (SP0770), 12V strike circuit + flyback diode.
- The backend /door/validate endpoint (§11) and the access-code.ts security fixes (§12).
- Egress/fire-safety obligations (§13).

This document replaces the **controller** (ESP32 -> Raspberry Pi 4) and the **firmware** (ESPHome -> a hardened Python service), and adds the **Pi reliability hardening** that the ESP32 didn't need.

2. Can a Raspberry Pi do the same job? - honest comparison

Yes, functionally identical. The Pi reads the keypad over GPIO, validates the code, and fires a relay to release the door - same as the ESP32. The differences are reliability, cost, and capability.

Aspect	ESP32 + ESPHome	Raspberry Pi 4 (hardened)
Operating system	None (firmware)	Full Linux - more to maintain
Boot/recovery	<1 s, instant after power loss	~25-40 s; needs hardening to survive power loss
#1 failure mode	(few)	SD-card corruption - must mitigate (SSD boot / read-only FS)
Power draw	~0.5-1 W	~3-6 W
HA integration	Native ESPHome API	MQTT / REST
Cost (controller side)	~\$30	~\$150-200 (Pi + PSU + storage + UPS)
Local compute	Limited	Full - local DB validation, touchscreen, camera/ANPR
Effort to reach "rock-solid"	Low (robust by default)	Higher (this §)

Verdict: For a *pure* keypad->relay door, the ESP32 is the better engineering choice - cheaper, simpler, and more reliable with no extra work. **Choose the Pi** if you want the door device to: validate fully offline against a local database, drive a touchscreen, add a camera/number-plate reader, or you already standardise your fleet on Pi. When hardened as below, a Pi is every bit as reliable - it just takes deliberate work to get there.

3. Architecture options with a Pi

- **Option A - Pi as a "dumb edge" (mirror the ESP32).** Pi runs a thin service that reports keypresses to Home Assistant over MQTT; HA validates and commands the relay. *Simplest, but wastes the Pi.*
- **Option B - Pi as a standalone smart controller (recommended).** Pi validates codes locally against a synced SQLite cache, fires the relay directly, and reports events to HA/Convex. **Works fully offline.** This is the only reason to prefer a Pi, so the build below uses Option B.

4. Bill of materials (Jaycar AU)

Shared parts are the same as the ESP32 build. Pi-specific items are marked **[Pi]**.

#	Item	Jaycar Cat.	Qty	Approx AUD	Notes
1	[Pi] Raspberry Pi 4B (2GB ample, 4GB headroom)	XC9102 / XC9100	1	verify	Controller. 2GB is plenty for this.
2	[Pi] Official 5.1V 3A USB-C power supply	XC9122	1	~\$25	A quality supply is essential for Pi stability.
3	[Pi] Boot storage - USB SSD (preferred) or high-endurance microSD	confirm	1	\$20-60	Boot from SSD to avoid SD-card corruption.
4	[Pi] Li-Ion power pack / UPS for Pi (clean shutdown)	XC9060	1	~\$40	Battery ride-through + triggers graceful shutdown.
5	[Pi] Prototyping HAT (GPIO -> screw terminals)	XC9040	1	~\$15	Clean, permanent GPIO breakout.
6	12-Key Numeric Keypad (3x4 matrix)	SP0770	1	~\$8	0-9, *, #. 7-wire matrix.
7	4-Channel Relay Module (or Pi relay HAT)	XC4441	1	~\$15	3.3V logic caveat - see §6.
8	Electric Door Strike 12VDC, Fail-Secure (narrow)	LA5077	1	~\$40	[DECISION] Fail-safe alt: LA5079/LA5081. See §13.
9	Non-Contact IR Exit Switch (Request-to-Exit)	LA5187	1	~\$30	Or reuse existing green exit button.
10	12V regulated PSU (>=2 A)	confirm	1	~\$30	Powers the strike. Size for inrush.
11	Flyback diode 1N4004 (strike coil)	ZR1004	1	~\$1	Across strike coil. Essential.
12	IP-rated enclosure, wire, ferrules, screw terminals	assorted	-	~\$30	Permanent install - no breadboard.

Note: the Pi does **not** need the ESP32 (XC3800) or the buck converter (XC4514) from the ESP32 build - the Pi has its own 5V supply. Net, the Pi build costs roughly **5-6x more** on the controller side.

5. * Reliability hardening (the part that makes a Pi trustworthy)

A Pi is only as reliable as its hardening. Do **all** of these - this is what closes the gap with the ESP32:

- 1. Boot from SSD, not SD card.** SD corruption on power loss is the #1 Pi field failure. Use a USB SSD (Pi 4 supports USB boot) or an industrial/high-endurance card as a fallback.
- 2. Read-only root filesystem (overlay).** `sudo raspi-config -> *Performance* -> *Overlay File System* -> enable`. Writes go to RAM and vanish on reboot, so power cuts can't corrupt the OS. Keep the code cache on a small writable partition (see §8).

- 3. Hardware watchdog.** Add to /boot/firmware/config.txt: dtparam=watchdog=on. Enable the systemd watchdog (RuntimeWatchdogSec=15 in /etc/systemd/system.conf). A hung Pi auto-reboots.
- 4. UPS + graceful shutdown.** The Li-Ion pack (XC9060) rides through mains blips; on low battery, trigger a clean shutdown so the OS closes cleanly.
- 5. systemd service with auto-restart** for the keypad app (Restart=always) so a crash self-heals.
- 6. Minimise writes:** disable swap (sudo dphys-swapfile swapoff), log to tmpfs/journald volatile.
- 7. Lock it down:** static IP, SSH key-only (no password), no auto-login, host firewall, no desktop (use Pi OS Lite).

6. Wiring - keypad, relay & exit to Pi GPIO

Pi GPIO is **3.3V**. The SP0770 3x4 keypad is a passive matrix (safe on 3.3V). Suggested **BCM** pin map (mounted on the XC9040 prototyping HAT):

Signal	Pi GPIO (BCM)	Signal	Pi GPIO (BCM)
Keypad Row 1	GPIO 5	Keypad Col 1	GPIO 26
Keypad Row 2	GPIO 6	Keypad Col 2	GPIO 16
Keypad Row 3	GPIO 13	Keypad Col 3	GPIO 20
Keypad Row 4	GPIO 19	Relay IN1 (strike)	GPIO 21
Exit switch (REX) in	GPIO 12	Status LED	GPIO 25

Power/strike path: 12V PSU -> relay common/NO contacts -> door strike -> back to PSU, with the **1N4004 flyback diode across the strike coil** (band/cathode to +12V). The Pi is powered by its **own 5V USB-C PSU** - do **not** back-feed it from the 12V rail.

[!] Relay logic-level caveat. Many 5V relay boards (incl. XC4441) want 5V on JD-VCC and expect a ~5V IN signal; the Pi only outputs 3.3V. Options: (a) power the relay-board **logic** at 3.3V if it supports it; (b) use an **opto-isolated** board with separate JD-VCC (5V) and drive IN from 3.3V (works on most); (c) safest - use a **purpose-built Pi relay HAT** with native 3.3V logic. Test the relay actually switches reliably from a 3.3V GPIO before final install.

Isolation: the relay **contacts** switch the 12V strike; the relay **coil** side is opto-isolated from the Pi. Keep the 12V door circuit physically separate from logic wiring. Never switch mains with this board.

7. OS setup (headless)

- Flash **Raspberry Pi OS Lite (64-bit)** to the SSD/card with Raspberry Pi Imager. In Imager's advanced settings: set hostname, **enable SSH with a public key**, set a static-IP-friendly user, configure Wi-Fi only if not on Ethernet (Ethernet preferred for a fixed device).
- First boot, then `sudo apt update` & `sudo apt full-upgrade`.
- Set a **static IP** (via your router DHCP reservation or /etc/dhcpd.conf).
- Apply the \$5 hardening (watchdog, overlay FS **last**, swap off).
- Install deps:

```
sudo apt install -y python3-pip python3-venv
python3 -m venv ~/keypad-venv && source ~/keypad-venv/bin/activate
pip install RPi.GPIO requests paho-mqtt
```

8. Software - the keypad service (Python, Option B)

Runs as a systemd service. Scans the keypad, validates locally first (offline-safe), pulses the relay, and reports to HA over MQTT. Keep the SQLite cache on a small **writable** partition (the rest of the FS is read-only).

```
#!/usr/bin/env python3
# /opt/keypad/keypad_service.py - starting point, review before production
import time, sqlite3, threading, requests, json
import RPi.GPIO as GPIO
import paho.mqtt.client as mqtt

ROWS, COLS = [5, 6, 13, 19], [26, 16, 20]
KEYS = [['1', '2', '3'], ['4', '5', '6'], ['7', '8', '9'], ['*', '0', '#']]
RELAY, EXIT_BTN, LED = 21, 12, 25
DB = "/var/keypad/codes.db" # writable partition
VALIDATE_URL = "https://<deployment>.convex.site/door/validate"
DOOR_TOKEN = "<per-device-secret>"
RELEASE_SECS, MAX_FAILS, LOCKOUT_SECS = 4, 5, 60

GPIO.setmode(GPIO.BCM)
for r in ROWS: GPIO.setup(r, GPIO.OUT, initial=GPIO.HIGH)
for c in COLS: GPIO.setup(c, GPIO.IN, pull_up_down=GPIO.PUD_UP)
GPIO.setup(RELAY, GPIO.OUT, initial=GPIO.LOW) # de-energised = locked (fail-secure)
GPIO.setup(LED, GPIO.OUT, initial=GPIO.LOW)
GPIO.setup(EXIT_BTN, GPIO.IN, pull_up_down=GPIO.PUD_UP)
```

```

def scan_key():
    for ri, r in enumerate(ROWS):
        GPIO.output(r, GPIO.LOW)
        for ci, c in enumerate(COLS):
            if GPIO.input(c) == GPIO.LOW:
                GPIO.output(r, GPIO.HIGH)
                return KEYS[ri][ci]
        GPIO.output(r, GPIO.HIGH)
    return None

def local_valid(code):
    # cache holds today's codes with start/end epoch; offline-safe
    now = time.time()
    con = sqlite3.connect(DB); cur = con.cursor()
    cur.execute("SELECT bay FROM codes WHERE code=? AND ?>=start AND ?<=end "
        "AND paid=1 AND cancelled=0", (code, now, now))
    row = cur.fetchone(); con.close()
    return row[0] if row else None

def remote_valid(code):
    try:
        r = requests.post(VALIDATE_URL, json={"code": code},
            headers={"X-Door-Token": DOOR_TOKEN}, timeout=3)
        d = r.json(); return d.get("bay") if d.get("allow") else None
    except Exception:
        return None # network down -> rely on local cache

def release(bay):
    GPIO.output(RELAY, GPIO.HIGH); GPIO.output(LED, GPIO.HIGH)
    publish_event("granted", bay)
    time.sleep(RELEASE_SECS)
    GPIO.output(RELAY, GPIO.LOW); GPIO.output(LED, GPIO.LOW)

def publish_event(result, bay=None):
    try:
        mqttc.publish("krickora/door/event",
            json.dumps({"result": result, "bay": bay, "ts": time.time()}))
    except Exception: pass

mqttc = mqtt.Client(); mqttc.connect("homeassistant.local", 1883, 60); mqttc.loop_start()

def sync_cache(): # pull today's valid codes from backend every few minutes
    while True:
        try:
            r = requests.get(VALIDATE_URL.replace("/validate", "/active-codes"),
                headers={"X-Door-Token": DOOR_TOKEN}, timeout=5)
            con = sqlite3.connect(DB); cur = con.cursor()
            cur.execute("CREATE TABLE IF NOT EXISTS codes(code TEXT PRIMARY KEY, "
                "bay TEXT, start REAL, end REAL, paid INT, cancelled INT)")
            cur.execute("DELETE FROM codes")
            for c in r.json().get("codes", []):
                cur.execute("INSERT OR REPLACE INTO codes VALUES(?,?,?,?,?,?)",
                    (c["code"], c["bay"], c["start"], c["end"], 1, 0))
            con.commit(); con.close()
        except Exception: pass
        time.sleep(180)

threading.Thread(target=sync_cache, daemon=True).start()

buf, fails, locked_until, last = "", 0, 0, 0
while True:
    k = scan_key()
    if k and (time.time()-last) > 0.25:
        last = time.time()
        if time.time() < locked_until:
            publish_event("locked_out"); buf = ""
        elif k == '*': buf = ""
        elif k == '#':
            bay = local_valid(buf) or remote_valid(buf)
            if bay: fails = 0; release(bay)
        else:
            fails += 1; publish_event("denied")
            if fails >= MAX_FAILS:
                locked_until = time.time()+LOCKOUT_SECS; fails = 0
                buf = ""
        else:
            buf = (buf + k)[:6]
    if GPIO.input(EXIT_BTN) == GPIO.LOW: # request-to-exit (if wired to controller)
        release("exit")
        time.sleep(0.02)

```

systemd unit /etc/systemd/system/keypad.service:

```

[Unit]
Description=Krickora keypad door service
After=network-online.target
[Service]
ExecStart=/home/pi/keypad-venv/bin/python /opt/keypad/keypad_service.py
Restart=always
RestartSec=3
WatchdogSec=30
[Install]
WantedBy=multi-user.target

```

```
sudo systemctl enable --now keypad.service
```

Why local-first validation: checking the SQLite cache before the network means the door **keeps working if the internet/Convex/HA is down** - the Pi's main advantage over the ESP32. The cache is refreshed every 3 minutes from the backend.

9. Home Assistant integration

The Pi publishes events to MQTT (krickora/door/event). In HA, add an MQTT sensor / automations to log entries, drive the booked bay (reuse the calendar-driven bay automations), and alert on locked_out/denied bursts. Optionally HA can publish back to krickora/door/cmd to remote-release for staff.

10. Offline / fail-safe behaviour [DECISION]

- **Internet down:** local SQLite cache validates today's codes -> entry still works.
- **HA/MQTT down:** door still works (validation is local); events queue/are dropped - decide if that's acceptable or needs buffering.
- **Mains down:** Li-Ion pack rides through; on low battery, graceful shutdown. On restore, the Pi reboots (~30 s) - **the door is briefly unmonitored during boot**; decide whether the strike fails locked (secure) during this window.
- **Power-on:** relay initialised LOW = de-energised = locked (fail-secure strike).

11. Backend - /door/validate + /door/active-codes (Convex)

These endpoints do not exist yet (http.ts has only health/auth/Stripe). Add:

```
POST /door/validate { "code":"4821" } -> { "allow":true,"bay":"bay_4" }
GET /door/active-codes -> { "codes":[{"code,bay,start,end}, ...] }
Headers: X-Door-Token: <per-device secret>
```

Server checks: code exists; now within [start-preGrace, end+postGrace];
paid & not cancelled; device token valid; rate-limit/lockout OK.

/active-codes returns only **today's** valid codes for the local cache (never the full history).

12. [!] Backend defects to fix first (unchanged from ESP32 plan)

src/lib/access-code.ts is not yet safe to drive a physical door:

- **In-memory Set (activeCodes)** resets on redeploy and isn't shared across instances -> invalidateAccessCode() and the 24h setTimeout won't reliably work. **Store codes on the booking row in the Convex DB**, indexed, validated server-side against the booking window.
- **4-digit = 9,000 combinations** -> brute-forceable. Enforce **rate-limiting + lockout** (the keypad service does the edge side; the server must too), keep codes valid **only during the booking window + grace**, consider 6-digit for new bookings.
- **Time-bind at validation** - never open outside the window.
- **Log & alert** on repeated failures.

13. [lock] Egress & fire-safety compliance - must confirm (unchanged)

A powered entry on an occupied facility carries building-code obligations (**NCC/BCA, AS 1428, fire egress**). People inside must **always** be able to exit regardless of power, network, or a Pi reboot.

- **Fail-secure vs fail-safe** is a safety decision - confirm against local code. Note the Pi's ~30 s boot window: egress must not depend on the controller being up.
- Provide **free mechanical egress** (handle/push-bar) and/or a fail-safe exit path independent of the electronics.
- **Licensed installer / building surveyor sign-off** before commissioning.

14. Installation steps

1. Flash Pi OS Lite to SSD; SSH-key + static IP; apt update.
2. Wire keypad + relay + exit on the XC9040 HAT per §6; **verify the relay switches from a 3.3V GPIO** before connecting the strike.
3. Deploy keypad_service.py + systemd unit; test the full chain on the bench (code -> validate -> relay pulse ->

- MQTT event).
4. Apply §5 hardening **last** (enable overlay FS, watchdog, swap off) and re-test after reboot.
 5. Fit the strike + 12V circuit with flyback diode; verify fail-secure/fail-safe matches the egress decision.
 6. Mount keypad (door face), Pi enclosure (inside), exit switch (egress side).
 7. Deploy backend /door/validate + /door/active-codes + DB code storage + rate-limiting (§11-12).
 8. Installer/surveyor egress sign-off -> go live.

15. Commissioning test checklist

Test	Expected
Valid code, within booking window	Door releases ~4 s; bay powers up; event logged
Valid code, outside booking window	Denied; logged
Wrong code x5	Lockout/cooldown (edge and server); alert raised
Internet pulled	Valid code still works via local SQLite cache
MQTT/HA pulled	Door still works; events handled per §10 decision
Mains pulled to Pi	Rides through on battery; graceful shutdown at low batt; recovers on restore
Pi forced reboot (watchdog)	Service auto-starts; door reachable within boot window; egress unaffected
Read-only FS check	Reboot loses no config; code cache persists on writable partition
Cancelled booking code	Denied after next cache sync (<=3 min) or immediately via remote check

16. Open decisions before ordering

1. **[DECISION]** Controller: **ESP32** (cheaper/simpler/robust - see keypad-build-plan.md) vs **Raspberry Pi** (this doc; choose only for offline-local-DB / touchscreen / camera).
2. **[DECISION]** Pi storage: USB SSD (recommended) vs high-endurance microSD.
3. **[DECISION]** Door hardware: electric strike (this plan) vs trigger existing roller-door operator vs both.
4. **[DECISION]** Single shared entry vs keypad per bay.
5. **[DECISION]** Fail-secure vs fail-safe (driven by §13 egress/compliance, incl. the Pi boot window).
6. **[DECISION]** Reuse existing green exit button vs new IR exit switch (LA5187).

17. Recommendation

If the goal is simply "reliable keypad entry," the **ESP32 build is the better choice** - cheaper, simpler, and robust with no hardening effort. **Choose this Raspberry Pi build** only if you specifically want **fully-offline local validation**, a **touchscreen**, or a **camera/number-plate reader** at the door - and commit to the §5 hardening so its reliability matches the ESP32. Both share the same strike, exit, backend, and compliance work.

Krickora keypad entry - Raspberry Pi build, draft for review. Verify all Jaycar cat. numbers, pricing and stock, validate the relay logic-level behaviour, and obtain licensed installer / building-surveyor sign-off on egress and fire compliance before installation. Example code is a starting point, not production-hardened. Indicative pricing only.